

Neutron Radiation Testing of Multiple System-on-Chip Devices Using a Unified Test Framework

Jeffrey Goeders*, Weston Smith*, Jacob Bertrand*, Ethan Hunter*, Maria Kastriotou†, Michael Wirthlin*

* Department of Electrical and Computer Engineering
Brigham Young University
Provo, UT, USA

† ISIS Facility, UKRI-STFC
Rutherford Appleton Laboratory
Didcot, OX11 0QX, United Kingdom

Abstract—This work presents neutron testing of various SoC devices using a unified test framework, which emphasizes measuring component-level failure rates. The framework is tested using four different SoC devices, with 16 boards in total.

I. MOTIVATION AND OVERVIEW

There is growing demand for using Commercial-off-the-Shelf (COTS) devices in space applications, and other high-radiation environments, due to their high performance and relatively low cost, compared to radiation-hardened devices. Modern system-on-Chip (SoC) devices are particularly attractive for these types of applications, as they provide a large range of computing resources in a single package, providing high performance along with low power consumption. The ever-increasing use of machine learning and other computationally intensive algorithms is further increasing the demand for using state-of-the-art SoC devices in these applications.

New SoC devices are being developed at a rapid pace, and continue to encompass more and more specialized components, such as cryptographic engines, machine learning accelerators, multi-level caching architectures, scratchpad memories, complex on-chip communication networks, and other components that are not typically found in traditional microcontrollers. These components often contain significant internal state, and may be susceptible to single-event effects (SEE).

While radiation testing of microcontrollers and processors is a well-studied topic, with significant existing

literature ([1]–[6] are just a few examples), many previous works have focused on testing a single component of the device, such as the processor cores, caches, or on-chip memories. However, as the complexity of the devices increases, the testing methodology must also evolve to keep pace. Several recent works, including our own recent work, have focused on creating test frameworks that emphasize exercising a larger set of the device’s components [7]–[10]. In our most recent work [11], we focused on developing a test methodology that would allow us to test multiple components of the SoC simultaneously, in order to maximize the amount of data we could collect in a given amount of beam time.

Designing complex test systems is time consuming, and requires hand-crafting test programs to exercise each component that is being tested. As new devices are developed, the testing process must be repeated, and the test programs must be re-written to accommodate the new device. Often much of the framework code can be reused when migrating to a new device, but depending on architectural differences, the test programs themselves may need to be re-written. These test frameworks are typically proprietary, and are not often shared with the community, which creates hurdles for new researchers, and results in duplication of efforts across the community.

In this work, we focused on porting our SoC test framework from one device family that we had tested multiple times previously (the AMD Zynq UltraScale+ MultiProcessor SoC (MPSoC)), to several new device families, including a STM32 microcontroller, a Raspberry Pi 3, and a PolarFire Icicle Kit board. This was primarily done to demonstrate the versatility of the parallelized test approach that we had done previously.

However, in porting our test framework to new device

The project or effort depicted was or is sponsored by the Department of the Defense, Defense Threat Reduction Agency under award HDTRA1-20-2-0002. The content of the information does not necessarily reflect the position or the policy of the federal government, and no official endorsement should be inferred.

types, we sought to create a single unified code base that could be used across all our devices. This was done to eliminate the need to re-write code when possible, but also to make it easier to test new devices in the future. The new devices we support represent a wide range of devices, with substantially different architectures, features, and programming models. Ultimately we were able to successfully use a single code base to test all four devices, and along the way, we learned several lessons that we hope will be useful to other researchers in the field. In the future, we hope to make our test framework open-source, and to share it with the community.

Our test framework was used to test the devices at the ChipIR facility at the STFC Rutherford Appleton Laboratory in November 2024. We tested all four devices simultaneously, with a total of 16 boards executing at once, many with multiple test programs running in parallel on different cores.

II. PARALLELIZED TESTING METHODOLOGY

We conducted neutron radiation testing on four different devices: 1) an STM32L433RC microcontroller on a NUCLEO-L433RC-P development board, 2) a Raspberry Pi 3B+, 3) a MicroChip PolarFire SoC on a Icicle Kit board, and 4) an AMD Zynq UltraScale+ MPSoC on an Ultra96v2 development board.

A primary goal of porting our test framework to new device families, was to demonstrate the versatility of our parallelized test approach. Our test approach, which is the focus of a previous publication [11], is designed to maximize the amount of data we can collect in a given amount of beam time. The general approach is to run multiple test programs simultaneously, either in a truly parallel fashion through leveraging multiple cores, or in a concurrent fashion by time-slicing a processing core, or both. Different test programs are designed to exercise and observe different components of the SoC. Some components, such as the processor cores, cryptographic engines, or Direct Memory Access (DMA) engines require active use to expose errors, while other components, such as various on-chip memories or the field programmable gate array (FPGA) configuration memory, will passively accumulate errors over time. Our framework is designed to construct program executables in such a way that this testing can be done strategically, for example, using one core to test passively accumulating components, while multiple other cores are used to test components that require active use.

In this work we demonstrated this approach on new device families. On the STM32 board, we tested the processor core and the Static Random Access Memory

(SRAM) memory in parallel. STM32 Board 1 and Board 2 both tested these components in a round robin fashion, alternating between executing multiple matrix multiplication operations and checking the SRAM against golden values. In contrast, STM32 Board 3 only tested the processor core, and STM32 Board 4 only tested the SRAM memory. This was done to collect results and compare the effectiveness of the parallelized approach to a more traditional approach of testing one component at a time.

On the Raspberry Pi 3B+ board, we were not able to test any components other than the processor core, due to limited documentation and the inability to run the device in a multi-core configuration.

On the PolarFire Icicle board, we successfully demonstrated our parallelized testing approach, with one core testing the processor core via matrix multiplication, another core testing the L2 cache via ECC status registers, and a third core testing the DMA engine.

The Ultra96v2 boards were largely a repeat of our previous work, verifying that the parallelized testing approach continued to work correctly after modifying the code base to support the other device families. On most boards one A53 core was dedicated to checking the On-Chip Memory (OCM), Double Data Rate (DDR), and Configuration RAM (CRAM) memories, while another core exercised the DMA engines, and the remaining A53 cores repeatedly exercised the processor cores and cryptographic engines. Each R5 core monitored its Tightly Coupled Memory (TCM) and performed matrix multiplication, in alternating fashion.

III. A UNIFIED TEST FRAMEWORK

Through porting our test framework to new device families, we learned several lessons that we believe will be useful to other researchers in the field. Some benefits of a unified code base:

- We reused module test code (matrix multiplication, memory fill and check) on multiple devices.
- We reused infrastructure code (logging, rate limiting, and test management) across all devices.
- A consistent output format across all devices allowed for a unified parsing system.
- We were able to use a single cmake-based build system for all devices, which allowed for a simplified build process and extensive continuous integration testing.

There were also several challenges we encountered along the way:

- Different vendors and device families have substantial differences in how they support multi-core bare metal execution.

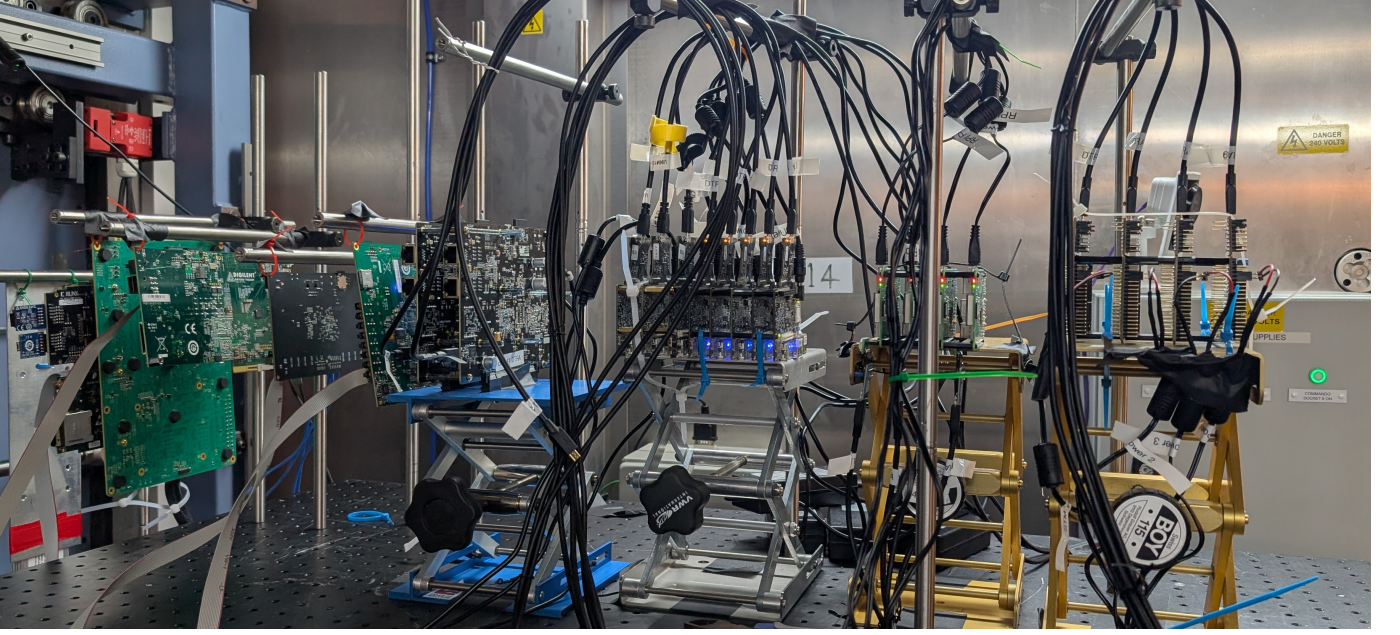


Fig. 1: Photograph of the test boards at the ChipIR facility. From right to left, the boards are: four STM32 boards, four Raspberry Pi 3 boards, eight Ultra96v2 boards (five were used for this work, and three are not part of this work), and three PolarFire icicle boards (two shown but the third was added after the photo). Additional boards to the left are not part of this work.

- 32-bit vs 64-bit architectures.
- Differing C standard libraries.
- Differing hardware meant that certain infrastructure code (timers, UART) required device-specific subclasses to handle these differences, while still maintaining a common interface.
- Devices varied in their memory models, for example, the PolarFire does not support synchronized heap usage in bare metal multi-core mode, while the Ultra96v2 does.

IV. RADIATION EXPERIMENT

We conducted a radiation test at the ChipIR facility at the STFC Rutherford Appleton Laboratory in November 2024. Figure 1 shows the test boards at the ChipIR facility.

Our test consisted of 16 total boards: four STM32 boards, four Raspberry Pi 3B+ boards, three MicroChip PolarFire icicle boards, and five AMD Ultra96v2 boards. The boards were monitored by a controller program running on the host computer, which would monitor the UART output of each board.

Upsets to memories were corrected by the test program, which would reinitialize the memory and continue running. In these cases the boards were not rebooted.

For other modules, such as the matrix multiplication processor tests, DMA, and the cryptographic engine

TABLE I: Test Results from ChipIr

Module	Upsets	Fluence	Cross Section (+/- 95% range)
STM32			
CPU (MxM)	3735	2.98e+12	1.25e-09 (1.21e-09–1.29e-09)
SRAM	4612	6.26e+12	5.62e-15 (5.45e-15–5.78e-15)
PolarFire			
CPU (MxM)	5	3.18e+12	1.57e-12 (5.03e-13–3.68e-12)
Cache-Data	5728	3.42e+11	9.99e-16 (9.73e-16–1.03e-15)
Cache-Meta	534	3.42e+11	1.56e-09 (1.43e-09–1.70e-09)
DMA	0	1.43e+12	7.01e-13 (7.01e-14–3.93e-12)
RPi3			
CPU (MxM)	12156	9.57e+11	1.27e-08 (1.25e-08–1.29e-08)
Ultra96v2			
A53 (MxM)	25	4.23e+12	5.91e-12 (3.83e-12–8.70e-12)
R5 (MxM)	21	2.11e+12	9.93e-12 (6.15e-12–1.51e-11)
CRYPTO	22	4.23e+12	5.20e-12 (3.26e-12–7.85e-12)
DMA	113	4.23e+12	2.67e-11 (2.17e-11–3.17e-11)
DDR	14	4.22e+12	3.32e-12 (1.82e-12–5.57e-12)
OCM	1206	4.22e+12	1.36e-16 (1.28e-16–1.44e-16)
TCM	1559	8.45e+12	1.76e-16 (1.67e-16–1.85e-16)
PCAP	39486	4.22e+12	2.07e-16 (2.04e-16–2.09e-16)
Cache	1440	4.58e+12	1.12e-15 (1.06e-15–1.17e-15)

tests, the test program would report the error, and the controller program would power cycle and reprogram the board.

Each self-test program prints regular heartbeat mes-

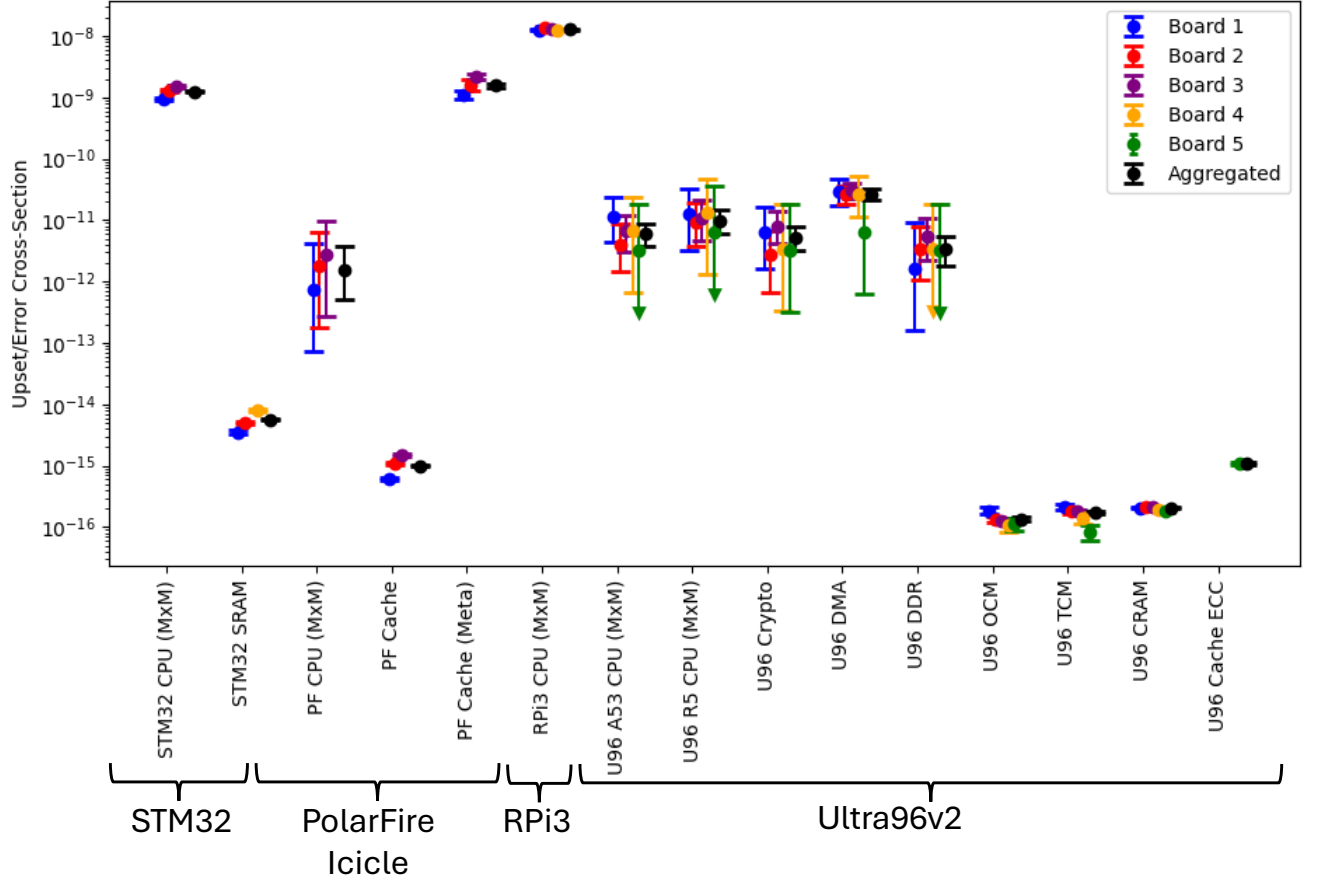


Fig. 2: Cross-section results of various SoC components. In cases where the tested component is a memory, and the total memory size is known, the cross-section is normalized to the size of the memory (STM32 SRAM, PolarFire L2 data cache, Ultra96v2 OCM, TCM, CRAM, L1 cache).

sages. If a message is not received for a certain amount of time, or a malformed message is received, the controller program would power cycle the board and reprogram it.

Table I provides the data for each system component that we tested. The data includes the number of upsets observed, the fluence accumulated across all boards that tested the module, and the computed cross-section. Start and stop times of each executable were recorded, and were matched against facility neutron counts to calculate the fluence for each module. Figure 2 shows the computed cross-sections with 95% confidence intervals. This figure provides a breakdown by each board that was tested, along with the aggregate results for each component, shown in black.

The results show we were successfully able to test multiple components of the SoC devices simultaneously, and collect component-level error rates from multiple devices in parallel. The PolarFire devices exhibited very low CPU cross-section, lower than the other three devices. The L2 ECC reporting successfully reported upsets in both the data and metadata (tag) sections. No

errors were detected using our PolarFire DMA test. The Raspberry Pi 3 devices were the softest of the devices tested, with the CPU encountering frequent errors when performing matrix multiplication.

In general, the STM32 results showed similar cross sections regardless if we are using the more aggressive parallelized approach (Board 1 and 2), or a dedicated approach (Board 3 and 4). The slightly higher cross section of the SRAM on Board 4 compared to Board 1 and 2 suggest that checking the SRAM more often may actually lead to an inflated cross-section, as errors in the CPU may be misinterpreted as SRAM upsets. Likewise, Board 3 exhibited slightly higher cross-section than Board 1 and 2. Again, we suspect this is due to Board 3 being fully dedicated to running the CPU test, which may expose more errors in the CPU than the case where both the CPU and SRAM are tested in round robin.

The Ultra96v2 boards showed a wide range of cross-sections, with results similar to what we observed previously in [11].

REFERENCES

- [1] H. Quinn, T. Fairbanks, J. L. Tripp, G. Duran, and B. Lopez, "Single-event effects in low-cost, low-power microprocessors," in *Radiation Effects Data Workshop*, Jul. 2014, pp. 246–254.
- [2] D. Hiemstra and A. Baril, "Single event upset characterization of the pentium(r) MMX and pentium(r) II microprocessors using proton irradiation," *IEEE Transactions on Nuclear Science*, vol. 46, no. 6, pp. 1453–1460, Dec. 1999.
- [3] S. E. Michalak, A. J. DuBois, C. B. Storlie, *et al.*, "Neutron beam testing of high performance computing hardware," in *Radiation Effects Data Workshop*, Jul. 2011, pp. 169–176.
- [4] D. A. Santos, L. M. Luza, M. Kastriotou, *et al.*, "Characterization of a RISC-v system-on-chip under neutron radiation," in *International Conference on Design & Technology of Integrated Systems in Nanoscale Era*, Jun. 2021, pp. 103–108.
- [5] S. M. Guertin and M. Cui, "SEE test results for the snapdragon 820," in *Radiation Effects Data Workshop*, Jul. 2017, pp. 155–160.
- [6] P. M. Aviles, A. Lindoso, J. A. Belloch, M. Garcia-Valderas, Y. Morilla, and L. Entrena, "Radiation testing of a multiprocessor macrosynchronized lockstep architecture with FreeRTOS," *IEEE Transactions on Nuclear Science*, vol. 69, no. 3, pp. 462–469, Mar. 2022.
- [7] P. Maillard, J. Arver, C. Smith, O. Ballan, M. J. Hart, and Y. P. Chen, "Test methodology neutron characterization of xilinx 16nm zynq® UltraScale+™ multi-processor system-on-chip (MPSoC)," in *Radiation Effects Data Workshop*, Jul. 2018, pp. 206–209.
- [8] P. Maillard, M. Hart, J. Barton, J. Arver, and C. Smith, "Neutron, 64 MeV proton & alpha single-event characterization of xilinx 16nm FinFET zynq® UltraScale+™ MPSoC," in *Radiation Effects Data Workshop*, Jul. 2017, pp. 139–143.
- [9] J. D. Anderson, J. C. Leavitt, and M. J. Wirthlin, "Neutron radiation beam results for the xilinx UltraScale+ MPSoC," in *Radiation Effects Data Workshop*, Jul. 2018, pp. 194–200.
- [10] W. Stirk, E. Poff, J. Smith, J. Goeters, and M. Wirthlin, "Comparison of neutron radiation testing approaches for a complex SoC," *IEEE Transactions on Nuclear Science*, vol. 70, no. 4, pp. 505–514, Apr. 2023.
- [11] J. Goeters, W. Smith, M. Kastriotou, and M. Wirthlin, "A parallelized neutron radiation testing technique to understand failures within a complex SoC," *IEEE Transactions on Nuclear Science*, 2025, Submitted, under review.